



Soft Web Intelligence with the J-CO Framework

Paolo Fosci  and Giuseppe Psaila  

University of Bergamo - DIGIP, Viale Marconi 5, 24044 Dalmine, BG, Italy
{paolo.fosci, giuseppe.psaila}@unibg.it

Abstract. In the last two decades a plethora of approaches have been proposed to perform Web Intelligence to discover useful knowledge over the World-Wide Web. However, variety and vastness of the Web are still making this task a hard challenge.

Nonetheless, the Web is evolving. An example is the advent of the *JSON* format as the practical standard for exchanging data over the Internet.

In our previous work, we proposed the concept of Soft Web Intelligence: it is a modern interpretation of Web Intelligence based on the current technological panorama, in which *JSON* data sets can be gathered and stored within *JSON* document stores and processed by means of Soft Computing so as to Soft Querying them. Soft Web Intelligence is enabled by the *J-CO* Framework, a software tool that is natively able to manage, soft-query and transform collections of *JSON* documents, located either in *NoSQL* repositories or over the Internet. The paper illustrates our vision by presenting a plausible case study based on a weekly-updated data set that reports COVID-19 cases in European Countries.

Keywords: Soft web intelligence · J-CO framework · Continued acquisition of *JSON* data sets from Web sources · Soft querying *JSON* data sets

1 Introduction

Web Intelligence [22] has been introduced two decades ago with the aim of developing tools for collecting and analyzing data from the World-Wide Web. Indeed, the original definition (proposed in [22]) said: “*Web Intelligence (WI) exploits Artificial Intelligence (AI) and advanced Information Technology (IT) on the Web and Internet*”.

Nonetheless, many works are still published on the topic, because the goal was (and still is) very ambitious and wide. Furthermore, the Web is evolving constantly on directions that often do not help the implementation of Web Intelligence tasks.

In spite of this general trend, a major event has occurred that could help the development of Web Intelligence solutions. This event is the advent of *JSON* (JavaScript Object Notation) as the practical standard for sharing data sets over the Internet, in place of XML. The consequent evolution and acceptance of “*JSON* document stores” (*NoSQL* databases that manage *JSON* data sets in a native way), has dramatically changed the technological panorama since [22]; thus, it is reasonable thinking about novel interpretations of the idea of Web Intelligence.

Fuzzy Logic and Soft Computing are undoubtedly considered methods and techniques for Artificial Intelligence. So, why not to use them for Web-Intelligence scopes,

for processing data sets represented as collections of *JSON* documents that are acquired from the Web and are stored within *JSON* document stores?

The idea came out while working on the *J-CO* Framework, a tool under development at University of Bergamo (Italy), specifically designed to perform complex integration and querying on *JSON* data sets; its query language, named *J-CO-QL⁺*, currently provides powerful soft-querying capabilities: soft conditions can be exploited, so as to express imprecise and vague selection conditions on *JSON* data sets.

Based on these considerations, a novel interpretation of Web Intelligence, named *Soft Web Intelligence*, was conceived in [11]. In this paper, we consolidate the vision and present a novel case study, by means of which we show how *Soft Web Intelligence* can be practically performed by means of a modern and stand-alone tool (the *J-CO* Framework) that opens the way to further developments for the future.

The paper is organized as follows. Section 2 provides the background from which the work has originated. Section 3 provides a consolidated vision of the concept of *Soft Web Intelligence*. Section 4 presents the current organization of the *J-CO* Framework, highlighting those components and capabilities that enable *Soft Web Intelligence*. Section 5 extensively presents a practical case study, in which we show how to conceive, set up and perform tasks in a scope of *Soft Web Intelligence*. Finally, Sect. 6 draws conclusions and future work.

2 Background

In this section, we present the background from which this work has originated.

The concept of Web Intelligence was introduced in [22]. Clearly, it originates from the concept of “Business Intelligence”, which encompasses methods and tools for reporting, making multi-dimensional analysis and, more in general, discovering knowledge from data possibly gathered in data warehouses and data marts [16]. In some sense, we can figure out that Web Intelligence is the evolution of Business Intelligence towards the World Wide Web.

The work [22] talks about Artificial Intelligence as the key tool for performing Web Intelligence, but what is Artificial Intelligence?

Even though many people think about Neural Networks and Deep Learning, when hearing the term Artificial Intelligence, also techniques for Data Mining are very popular techniques that belong to the area of Artificial Intelligence. Consequently, their application to Web Intelligence has been immediately recognized [12], together with the consciousness that peculiarities of the Web constituted (and still constitute) a hard challenge, if compared to their application on flat data. The work [28] provides an interesting application of Web Intelligence to Digital Libraries, being mainly focused on link analysis and Web-log analysis.

Fuzzy Logic and Soft Computing are considered as a side part of Artificial Intelligence. Indeed, Zadeh, the inventor of Fuzzy-Set Theory [23], provided his own interpretation for the concept of Web Intelligence in [24, 25]. Specifically, Zadeh thought about the concept of “WebIQ” or “WIQ”, as the evolution of the idea of “Machine IQ”: machines become more and more intelligent as far as their capability to answer vague or imprecise questions is concerned; Fuzzy-Set Theory provides the formal tools towards

WebIQ. Another remarkable interpretation of the concept of Web Intelligence is “Computational Web Intelligence” [26], i.e., the adoption of “Computational Intelligence” to Web Intelligence scopes. We also mention “Brain Informatics” [27], whose goal is to foster Web Intelligence through techniques that come out from studying human brain.

It is worth noticing that, at the best of our knowledge, in the literature there are very few papers that are explicitly focused on the adoption of fuzzy logic and soft computing in Web-Intelligence scopes. We found the paper [13], in which authors exploit soft computing in a group decision-making system to express preferences, while Web Intelligence is used to gather knowledge to be exploited to make decisions. Another related paper is [17], in which the authors propose to use FUZZYALGOL, a fuzzy procedural programming language [21], to perform soft querying on Web sources.

As far as data formats are concerned, in the two passed decades since [22], the major event has been the advent of *JSON* as *de-facto* standard format for information interchange over the Internet, in place of *XML*. As a direct consequence, in the area of *NoSQL* databases, “*JSON* document stores” such as *MongoDB* have been proposed and have become very popular. These consideration motivated the development of the *J-CO* Framework, i.e., developing a general-purpose tool for performing complex manipulation and querying of *JSON* data sets [2]. The original capabilities reported in [2, 19] have been extended with soft-querying capabilities [7–11]. Furthermore, the *J-CO* Framework has been exploited as an execution engine for a domain-specific language for soft querying spatial features in *GeoJSON* documents [6, 20].

Currently, the *J-CO* Framework is a unique tool in the panorama of the few proposals for exploiting Fuzzy-Set Theory in the world of *JSON* data management. Specifically, [1, 15] proposed *fMQL*, an extension of *MQL* (the *MongoDB* query language). The extension is based on *JSON* documents previously tagged with “fuzzy labels”. In contrast, [14] proposed an extension of the *MongoDB* data model to support fuzzy values at the level of single document field. Definitely, they propose a fuzzy *JSON* document store, which was implemented on top of *MongoDB*.

At the best of our knowledge, no other proposals are somehow related to the *J-CO* Framework.

Consequently, we can say that our vision of *Soft Web Intelligence*, which will be introduced in Sect. 3, can be considered as a modern interpretation of the original idea of Web Intelligence, on the basis of the current scientific and technological panorama.

3 Vision: Soft Web Intelligence

Is there a uniform vision of Web Intelligence? Based on the literature (that we summarized in Sect. 2), the answer is clearly “no, there is not a uniform vision of Web Intelligence”. Probably this is due to the fact that different problems ask for different solutions and conduct to different visions.

In [11], we proposed the notion of *Soft Web Intelligence*, thinking about scopes in which it is necessary to collect *JSON* data sets from Web sources, so as to query them in a vague or imprecise way and obtain useful knowledge.

Our vision characterizes these scopes in a general way, regardless of the specific data sets to acquire and process, in which soft computing and soft querying can signif-

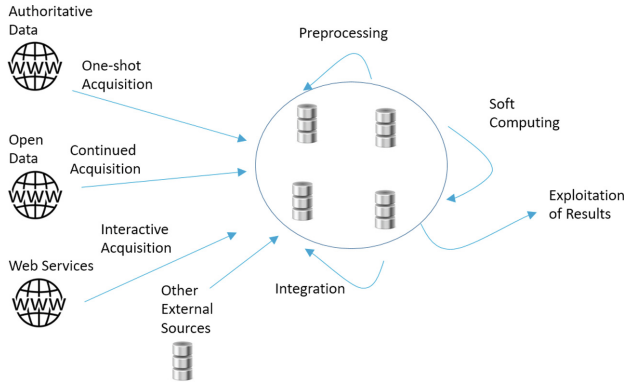


Fig. 1. Vision of Soft Web Intelligence.

icantly help analysts to find what they are looking for from web sources. The vision is depicted in Fig. 1; hereafter, we present it.

– **Web Sources.** In *Soft Web Intelligence*, data come from many Web sources. In order to understand how to deal with these sources, we provide a classification of them.

- **Authoritative Data.** Public Administrations and Authorities often publish data that can be considered “immutable” or “slowly mutable”. As an example, the reader can think about borders of municipalities, regions, and so on: they do not change or, if they change, this happens rarely and is due to the approval of acts by governments and parliaments.

Practically, this means that authoritative data sets can be downloaded only once and stored within databases. As far as their format is concerned, *JSON* is gaining more and more popularity. Considering again the case of administrative borders, they are often provided as *GeoJSON* data sets, which is a specific *JSON* format for representing geographical information layers [5].

- **Open-Data Portals.** Open-Data portals are a channel commonly used by public administrations to distribute data sets that concern the administered territory, so as to achieve the requirement of “transparency” (this is why they are said “open”).

On Open-Data portals, it is possible to find data sets that change continuously and it could be of interest for the analyst not to lose the previous versions; in such a case, it is necessary to perform a “continued acquisition” of these data sets, to accumulate all versions that change in time.

As far as the format of data is concerned, among the others *JSON* is becoming more and more adopted.

- **Web Services.** Often, useful data are provided by Web Services in a way that it is not possible to obtain them with one single call. For example, this is the case of “geo-coding”: provided latitude and longitude of a point on the Earth surface, a web service for geo-coding returns the address of the point; clearly, this kind of service asks for as many calls as the number of points to geo-code.

As far as the format of data provided by web services, we can say that substantially all of them provide data as *JSON* documents.

- **Data Storage.** In our vision, the *Data Storage Area* (the blue circle in Fig. 1), encompasses systems for storing data sets to integrate and process. Since we are considering scopes in which data sets are provided as *JSON* documents, *NoSQL* databases must be exploited; in particular, “*JSON* document stores” (or, simply, “*JSON* stores”) have imposed on the panorama of DBMSs (DataBase Management Systems), due to their ability to store and query *JSON* documents in a native way.
- **External Data.** Any kind of external data (i.e., not directly acquired from Web sources) can provide valuable information. Clearly, since we are considering a scope where data are represented as *JSON* data sets and stored in *JSON* stores, external data should be represented as *JSON* data sets as well.
- **Processing Tasks.** Many processing tasks could be figured out. In Fig. 1, we report the types that we consider most relevant (without excluding any other processing tasks, in principle).
 - **Pre-processing.** Once downloaded, a data set usually needs to be pre-processed, so as to clean it and change the format, to make it suitable for further processing. If data to be pre-processed are obtained through continued acquisition, their format is known and pre-processing could be automated.
 - **Integration.** Once data sets are collected, they could be integrated. Integration could be either an independent task or a part of pre-processing; this latter situation occurs when it must be performed on data sets obtained through continued acquisition, whose structure is known in advance.
 - **Soft Computing.** Once data sets to analyze have been built, it is possible to perform tasks of “soft computing”. Here, a plethora of techniques can be used in principle, such as Neural Networks; indeed, many techniques that fall into the area of Artificial Intelligence (AI) are also considered Soft-Computing techniques. A specific category of these techniques is devoted to “Soft Querying”: by relying on Fuzzy-Set Theory, they allow for specifying “Soft Conditions”, i.e., conditions whose evaluation does not provide a crisp “true” or “false” value, but a “satisfaction degree” in the $[0, 1]$ range. This way, items that do not exactly match conditions can still emerge, possibly providing analysts with unexpected results. In this paper, we focus on soft querying, but the vision is open to any kind of soft-computing technique.

The next section presents the *J-CO* Framework, which currently provides the technical capabilities to build a scope of *Soft Web Intelligence*. Indeed, we conceived the concept of *Soft Web Intelligence* while introducing novel capabilities into the *J-CO* Framework based on the needs of our colleagues working on Geography (see [3,4]).

4 The J-CO Framework

In this section, we provide a synthetic presentation of the *J-CO* Framework, by highlighting the features that were specifically introduced in [11] to support the vision of *Soft Web Intelligence*.

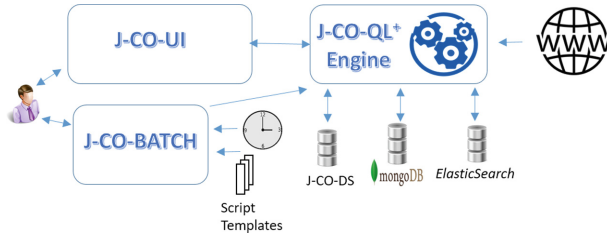


Fig. 2. Organization of the *J-CO* Framework.

4.1 Organization of the Framework

The *J-CO* Framework is designed to provide data engineers and data analysts with a powerful support for gathering, integrating and querying possibly-large collections of *JSON* data sets. The framework is built around *J-CO-QL⁺*, a query language for specifying high-level transformations of collections of *JSON* documents in a declarative way. The language will be presented in action in Sect. 5.

Figure 2 depicts the tools that constitute the framework. We briefly introduce them hereafter.

- *J-CO-QL⁺ Engine*. This component actually executes *J-CO-QL⁺* scripts (i.e., queries). It is able to retrieve data from external document databases (for example, managed by *MongoDB*) and save results into them; it also can send HTTP requests to web services and portals to get *JSON* data sets directly from Web sources.
- *J-CO-DS*. This component is a novel document store specifically designed to store large or very large single documents (such as many *GeoJSON* documents), so as to overcome limitations of other document databases (such as *MongoDB*). Section 4.2 is devoted to present this component.
- *J-CO-BATCH* is an off-line executor of *J-CO-QL⁺* scripts. It has been introduced in [11] to enable scheduled and/or parameterized execution of scripts. Section 4.3 is devoted to present this component.
- *J-CO-UI*. This is the user interface, by means of which analysts can write *J-CO-QL⁺* scripts, submit them to the *J-CO-QL⁺ Engine* and inspect results.

In the remainder of this section, we present the *J-CO-DS* and the *J-CO-BATCH* components in details.

4.2 *J-CO-DS*

As anticipated, *J-CO-DS* is a *JSON* store which is able to store very large documents [18]. It was specifically introduced in the *J-CO* Framework to overcome practical limitations provided by other *JSON* stores, such as *MongoDB*: indeed, *MongoDB* is not able to store large documents whose internal representation (named *BSON*) is greater than 16 MByte; furthermore, *MongoDB* adds an extra field named `_id` to documents, which alter *JSON* documents from the original ones.

J-CO-DS does not provide any computational capabilities, such as a query language, because it is an integral part of the *J-CO* Framework, so, queries can be performed by means of *J-CO-QL*⁺ and its engine.

Although *J-CO-DS* organizes a database as a set of “collections” of *JSON* documents, as *MongoDB*, it provides three different types of collection.

Static collections are the classical collections, i.e., created and possibly updated either by the user or by a *J-CO-QL*⁺ script.

Dynamic collections are thought for continued acquisition (they have been introduced in [11]); they are created specifying a pool of Web sources; the content of the collection is periodically updated with novel documents.

Let us define a dynamic collection in more details.

- A “dynamic-source descriptor” is a tuple

$$dsd = \langle name, URL, frequency, mode \rangle$$

in which many properties are defined. The *name* member is the name provided by the user to characterize the source. The *URL* member is the URL to contact to get documents. The *frequency* member is the frequency with which gathering must be repeated; admitted values can be expressed in hours (e.g. 2H), days (e.g. 4D) or weeks (e.g. 5W). Finally, the *mode* member denotes the way to update the content of the collection; its value can be either ‘*overwrite*’, i.e., only the last image of the Web source is kept, or ‘*append*’, i.e., a new image of the Web source is appended to those already present in the collection.

- Thus, a dynamic collection *dc* is described by the tuple

$$dc = \langle name, sources \rangle$$

where *name* uniquely identifies the collection within the database, while *sources* is a non-empty array of dynamic-source descriptors.

Notice that different sources could be accessed with different frequencies, depending on the frequency they provide novel data. Furthermore, for each source a different update mode can be specified. To understand this choice, we briefly illustrate how the dynamic collection works.

- Data collected from each source are managed in a dedicated storage area, so as to be managed independently of data coming from the other sources.
- A Web source usually provides either a single *JSON* document, or a *JSON* array (usually an array of documents, but not necessarily).

For each single request sent to the Web source that returns data (either a single document or an array), a *JSON* document is created, with the following fields: *source* is the name of the source (the *name* field in the *dsd* source descriptor); *url* is the contacted URL; *timestamp* is the acquisition time; *data* actually contains the returned data (either a single *JSON* document or an array).

This new document is inserted into the data space of the web source.

- In the case the `''overwrite''` update mode is selected, the previous content of the data space is cleaned and the new document is added. This means that only one document at a time is present in the data space.
In the case the `''append''` update mode is selected, the new document is added to the previous ones possibly present in the data space.

The rationale behind the two update modes is the following. The `''append''` mode is suitable when the goal is to keep all images of the Web Source. The usefulness of the `''overwrite''` mode is to decouple the $J\text{-CO-QL}^+$ batches and scripts from the Web source and the intermediate network connection that provides the data to process: this way, data can be easily processed, independent of the availability of the Web source and of the Internet connection.

Virtual collections provide a unified abstraction to many Web sources (they have been introduced in [11]). They are defined by specifying a pool of URLs, which provide collections of *JSON* documents. When a user or a $J\text{-CO-QL}^+$ script gets the virtual collection, the actual content is obtained by calling all associated services on-the-fly. In virtual collections, a source descriptor is a tuple

$$vsd = \langle name, URL \rangle$$

where only the *name* and *URL* members are present (it does not make sense to think about update mode or update frequency, because Web-source images are acquired on the fly).

A virtual collection is, in turn, defined by the tuple

$$vc = \langle name, sources \rangle$$

where *name* uniquely identifies the collection in the database, while *sources* is a non-empty array of virtual-source descriptors *vsd*.

Dynamic and Virtual collections can be altered only through the managing interface of $J\text{-CO-DS}$; they cannot be updated by $J\text{-CO-QL}^+$ scripts.

4.3 J-CO-BATCH

$J\text{-CO-BATCH}$ has been introduced in [11]. Its goal is to provide a “batch execution” of $J\text{-CO-QL}^+$ scripts. Hereafter, we present the functionality it provides.

- **Template Execution.** A *Template* is a $J\text{-CO-QL}^+$ script that is provided with “macro-parameter’s”. Their lexical structure is `##parameterName##`, since $J\text{-CO-QL}^+$ has no similar lexical elements.

In order to execute a template *t*, a context *C* must be provided; *C* is a key/value map, by means of which a macro-substitution of all macro-parameters *mp* $\in t$, with the corresponding value *C*(*mp*), is performed; if all macro-parameters in *t* have a corresponding value in *C*, the *t* template is actually transformed into an executable $J\text{-CO-QL}^+$ script.

Only constant values (either numbers or strings enclosed within single or double quotes), identifiers and path-expressions are valid values for macro-parameters in *C*.

- **Batch Execution.** Users can launch the execution of templates in batch mode, by providing the C context as a “property file”, as illustrated in Listing 5.
- **Scheduled Batch Execution.** Batch execution of template can be scheduled, either as a “one shot” execution, in which the template is executed at a given predefined instant, or as a “repeated” execution, to execute a template multiple times.

5 Soft Web Intelligence: A Practical Case Study

The ultimate goal of this paper is to show a practical case study that is in the scope of *Soft Web Intelligence*. By means of it, it will be clear our perspective.

5.1 Case Study

The recent COVID-19 pandemic provides a very interesting application of the idea of *Soft Web Intelligence*. Hereafter, we present the application context.

- The European Union (EU) has asked its Member States to weekly provide data about the number of official COVID-19 cases and the number of deaths. Currently (April 2023) these data are still communicated to the European Union.
- On the Open-Data portal of the European Centre for Disease Prevention and Control (ECDC)¹, a specific data set with the latest version of the data set is published, as a *JSON* array. This array contains all weekly updates for each country; consequently, it would be possible for an analyst to perform an historical study of the pandemic for European countries.
- Each item in this data set refers to a specific country by means of its country code. If it were necessary to add an extra piece of information, such as the name of the country capital, it would be necessary to look up for it on external web services that provide such a kind of information. *GeoNames*², a geographical database accessible from the Internet, provides a pool of web services for this purpose.
- There are several formats for country codes. ISO³, the International Organization for Standardization, defines two standard formats, named *alpha-2* and *alpha-3*, which use, respectively, two and three letters to identify a country. Thus, an analyst must be aware that ECDC identifies countries by means of an ISO *alpha-3* code, while *GeoNames* uses an ISO *alpha-2* code.

We foresee that the analyst would like to address the problem presented hereafter.

Problem 1. *Based on the data set providing the weekly counts for COVID-19 cases, look for those EU countries that, in the last week, could show an alert because the cases are too high in relation with the country population; for each of those countries, report also the name of its capital.*

Referring to the vision depicted in Fig. 1, Problem 1 is clearly a problem of *Soft Web Intelligence*, as illustrated in Fig. 3. It can be effectively managed by the *J-CO* Framework, as illustrated hereafter.

¹ ECDC web site: <https://www.ecdc.europa.eu/en>).

² GeoNames web site: <https://www.geonames.org/>.

³ ISO, country code web site: <https://www.iso.org/iso-3166-country-codes.html>.

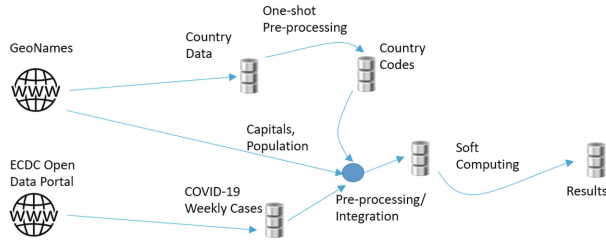


Fig. 3. Depicting the case study.

- A data set containing country data is provided by a *GeoNames* web service as a collection of *JSON* documents; this collection is stored in a *JSON* document store for later exploitation. However, in order to obtain a mapping between *ISO alpha-2* and *ISO alpha-3* country codes (and remove all unnecessary data) a one-shot pre-processing activity must be performed, so as to obtain only country codes. This task can be easily done by means of *J-CO-QL⁺*.
- Weekly updates of the data sets describing COVID-19 cases are made from the ECDC Open-Data portal. A dynamic collection managed by *J-CO-DS* is automatically filled in with the last image of the data set; this way, subsequent steps can be performed in an off-line manner, with respect to the Open-Data portal.
- The data set describing COVID-19 weekly cases can be automatically pre-processed so as to integrate it with country codes and with further data concerning each single country, such as the name of its capital and its population, if this latter information is missing. Again, a web service exposed by *GeoNames* provides this pieces of information.
- Finally, the soft query is performed, by running a *J-CO-QL⁺* template through *J-CO-BATCH*. The final collection contains the data of interest and can be later accessed for knowledge exploitation and dissemination.

In the following Sections, we present each single step in details.

5.2 Defining a Dynamic Collection Gathering Data About COVID-19 Pandemic

In order to collect the latest version of the ECDC data about COVID-19 pandemic, a database managed by *J-CO-DS* is created. In this database, a dynamic collection (see Sect. 4.2) is created, so that it acquires the latest version of the data set locally.

Listing 1 shows the commands issued to *J-CO-DS* to create the database and the virtual collection.

- Line 1 in Listing 1 creates the database `softIntelligenceDb`.

```

{
  "country"           : "Luxembourg",
  "countryCodeIsoAlpha2" : "LU",
  "countryCodeIsoAlpha3" : "LUX"
}

```

Fig. 4. Example of document in the static `countryCodes` collection.

- Line 2 creates the dynamic collection, with name `euOfficialCovidData`; notice the `--url` parameter, which specifies the URL to contact to get the collection on the ECDC Open-Data portal.
- To complete the definition of the dynamic collection, Line 3 sets the update frequency for the dynamic collection to one week (denoted as `1W`), since the frequency default value is one day. Then, Line 4 sets the update mode of the dynamic collection as `type 1`, that means “overwrite” mode, i.e., the new image of the Web source replaces the previous one (the default update mode is `type 0`, i.e., “append” mode).

This way, *J-CO-DS* starts getting the last state of the data set each week. The reader can notice that this way it is possible to have a clear list of data sets that are acquired from external sources. As an effect, the World Wide Web is actually viewed as a giant data store.

The `softIntelligenceDb` database holds also a static collection, named `countryCodes`; a document in this collection reports, for each European country, both the ISO *alpha-2* and the ISO *alpha-3* codes. Figure 4 shows an example document related to Luxembourg. The `countryCodes` collection is obtained by a previous acquisition activity: these codes are made available by a GeoNames web service⁴, together with other information; a *J-CO-QL*⁺ script, that we do not show for the sake of space, was executed to transform the data in the format shown in Fig. 4. Since country codes are static, they can be stored within a static collection, which is ready for further processing activities.

Listing 1. *J-CO-DS* commands: getting EU data.

```

1. create-database --name softIntelligenceDb
2. create-dynamic-collection --database softIntelligenceDb
   --collection euOfficialCovidData
   --url https://opendata.ecdc.europa.eu/covid19/nationalcasedeath/json/
3. set-frequency --database softIntelligenceDb --collection euOfficialCovidData
   --index 0 --frequency 1W
4. set-update-type --database softIntelligenceDb
   --collection euOfficialCovidData --index 0 --type 1

```

⁴ GeoNames web service to get country codes:

<http://api.geonames.org/countryInfoJSON?formatted=true&username=paolofosci>.

5.3 Repeated Scheduled Pre-Processing

Listing 2. *J-CO-QL*⁺ script: preprocessing EU COVID-19 data - Part 1.

```

1. USE DB softIntelligenceDb
   ON SERVER jcodes 'http://127.0.0.1:17017';

2. GET COLLECTION euOfficialCovidData@softIntelligenceDb;

3. EXPAND
   UNPACK WITH .data
   ARRAY .data TO .countryData;

4. SAVE AS euCovidDataUnpacked;

5. GROUP
   PARTITION WITH .countryData.item.year_week,
                  .countryData.item.country
   BY .countryData.item.country, .countryData.item.indicator
   INTO .array
   GENERATE
   BUILD {
     .country      : .countryData.item.country,
     .indicator    : .countryData.item.indicator,
     .year_week    : MAX_ARRAY (.array, STRING,
                               .countryData.item.year_week)
   };

6. JOIN OF COLLECTIONS temporary AS c1,
   euCovidDataUnpacked AS c2
   CASE
   WHERE
     .c1.country    = .c2.countryData.item.country AND
     .c1.indicator  = .c2.countryData.item.indicator AND
     .c1.year_week  = .c2.countryData.item.year_week AND
     .c1.indicator  = "cases"
   GENERATE
   BUILD {
     .country      : .c1.country,
     .indicator    : .c1.indicator,
     .yearWeek     : .c1.year_week,
     .countryCode  : .c2.countryData.item.country_code,
     .population   : .c2.countryData.item.population,
     .cases        : .c2.countryData.item.weekly_count
   };

```

In order to pre-process the data about COVID-19 pandemic, so as to extract and complete *JSON* documents concerning the last week, a *J-CO-QL*⁺ script is run by *J-CO-BATCH*, which is the engine that executes *J-CO-QL*⁺ scripts in a scheduled way (see Sect. 4.3). The script is reported in Listings 2 and 3, due to its length. Hereafter, we present it in details: this way, we also present the *J-CO-QL*⁺ language in action.

Selecting Data about the Last Week. The first part of the script (reported in Listing 2) actually extracts, from the euOfficialCovidData dynamic collection in the softIntelligenceDb database, the *JSON* collection that actually describes COVID-19 cases in the last communicated week. A series of steps is necessary to achieve this (apparently simple) goal.

- The **USE DB** instruction on Line 1 actually connects the execution process to the database; this way, in the remainder of the script, the softIntelligenceDb database will be visible to instructions.

```

{
  "timestamp" : "2023-04-07T03:59:25.855",
  "url"       : "https://opendata.ecdc.europa.eu/covid19/.../...",
  "data"      : [
    {
      "country"       : "Austria",
      "country_code"  : "AUT",
      "continent"     : "Europe",
      "population"    : 8978929,
      "indicator"     : "deaths",
      "year_week"     : "2020-01",
      "source"        : "TESSy COVID-19",
      "note"          : ""
    },
    ..., ... // other country documents
  ]
}

```

Fig. 5. Fragment of COVID-19 collection retrieved on line 2 in Listing 2.

```

{
  "timestamp" : "2023-04-07T03:59:25.855",
  "url"       : "https://opendata.ecdc.europa.eu/covid19/.../..."
  "countryData" : {
    "position" : 6930,
    "item" : {
      "continent" : "Europe",
      "country"   : "Luxembourg",
      "country_code" : "LUX",
      "cumulative_count" : 290244,
      "indicator" : "cases",
      "note" : "",
      "population" : 645397,
      "rate_14_day" : 1745.2824,
      "source" : "TESSy COVID-19",
      "weekly_count" : 5981,
      "year_week" : "2022-25"
    }
  }
}

```

Fig. 6. Example of document in the temporary collection generated by Line 3 in Listing 2, extracted from the document in Fig. 5.

- The GET COLLECTION instruction on Line 2 actually acquires the content of the euOfficialCovidData dynamic collection in the softIntelligenceDb database.

Figure 5 depicts a fragment of this collection. Notice that dynamic collections in *J-CO-DS* contain one document for each monitored URL; since we associated only one URL and for it only the last version is kept (see Listing 1), only one single doc-

ument is stored and thus retrieved by the `GET COLLECTION` instruction. Specifically, notice that the actual collection acquired from the ECDC Open-Data portal is contained within the `data` array field. In Fig. 5 we report two documents, one concerned with Austria and one concerned with Luxembourg. Notice that the fields contained in the two documents are not exactly the same, i.e., documents can be heterogeneous.

The collection produced by the instruction becomes the new *temporary collection* of the process, ready to become the input to the next instruction (see [19]).

- From Line 3 a pool of instructions extracts the most recent *JSON* documents from the temporary collection generated by Line 2. The `EXPAND` instruction on Line 3 unnests all the *JSON* documents in the `data` field of the lonely document in the temporary collection depicted in Fig. 5; Fig. 6 depicts one unnested document, specifically the one depicted in Fig. 5 related to Luxembourg.

Specifically, in Fig. 6, notice that in place of the source `data` array, there is a `countryData` field that contains a complex sub-document with two fields: the `position` field reports the position formerly occupied by the document in the source array; the `item` field actually contains the unnested document. Outside the `data` field, all fields previously present in the source global document are reported.

- Line 4 temporarily saves the current temporary collection into the *Intermediate-Results Database* (or *IRDB*): this is a private database for the query process under execution, within which it is possible to save temporary results to be exploited later in the same query process. Although the temporary collection is saved into the database, it is not affected and it survives as temporary collection of the query process.

In Line 4, notice that the *IRDB* is implicitly referred, in that the collection name is not followed by `@<dbname>` (as in Line 2).

- The goal of Line 5 is to discover, for each country, the most recent week for which data are reported. In Fig. 6, within the `item` field, notice the `year_week` field; its value is a string with the format `yyyy-wk`, where `yyyy` denotes the year, while `wk` is the number of week in the year.

To obtain documents like the one reported in Fig. 7 for Luxembourg, the `GROUP` instruction on Line 5 behaves as follows. (i) The `PARTITION` clause selects all documents in the input temporary collection having the necessary fields. (ii) The `BY` clause specifies the “grouping fields”, specifically the country name and the indicator. For each group of documents having the same values for the grouping fields, a unique document is created, having all the grouping fields and an array field whose name is specified by the `INTO` clause (in this case, the name is `array`), containing all the input grouped documents.

The `GENERATE` clause specifies a kind of “post-processing” of the generated documents, so as to prepare them to be inserted into the output temporary collection. On Line 5, the `BUILD` block actually projects the documents on the grouping field and adds the `year_week` field, whose value is obtained by the built-in `MAX_ARRAY` function, which is used to extract the maximum week identifier from within the documents in the `array` field. An example of final document for Luxembourg related to the indicator named “cases” is depicted in Fig. 7.

- Finally (for Listing 2), the temporary collection generated by Line 5 is exploited to select only the documents of interests from within the collection temporarily saved in the *IRDB*, which contains all the documents unnested from the official data set reported in Fig. 5.

Specifically, the `JOIN OF COLLECTIONS` instruction on Line 6 joins documents in the current temporary collection (see Fig. 6) and in the novel collection named `euCovidDataUnpacked`, which is saved within the *IRDB* (see Fig. 7); the former is aliased as `c1`, while the latter is aliased as `c2`. All pairs of documents $d_1 \in c1$ and $d_2 \in c2$ are considered, building, for each (d_1, d_2) pair, a novel *od* document with two fields, named `c1` and `c2`; d_1 is the value of the `c1` field, while d_2 is the value of the `c2` field.

Each *od* document is processed by the `CASE` clause, in which the `WHERE` selection condition selects those documents that are actually of interest; specifically, the condition selects those *od* documents that are obtained by pairing a document in `c1`, denoting the last week identifier for a country and an indicator, with the corresponding document in `c2`. This way, only the most recent documents for each country are actually obtained as `c2` fields.

For all these *od* documents, the `GENERATE` clause actually restructures the output documents, so as to flatten them, as reported in Fig. 8.

```
{
  "country"      : "Luxembourg",
  "indicator"    : "cases",
  "year_week"    : "2023-13"
}
```

Fig. 7. Example of document after Line 5 in Listing 2.

```
{
  "cases"        : 1011,
  "country"      : "Luxembourg",
  "countryCode"  : "LUX",
  "indicator"    : "cases",
  "population"   : 645397,
  "yearWeek"     : "2023-13"
}
```

Fig. 8. Example of document in the temporary collection generated by Line 6 in Listing 2.

Completing Documents with External Data. Listing 3 reports the second part of the scheduled *J-CO-QL*⁺ script that pre-processes data about COVID-19 pandemic.

Listing 3. *J-CO-QL⁺* script: preprocessing EU COVID-19 data - Part 2.

```

7. JOIN OF COLLECTIONS temporary AS c1,
                        countryCodes@softIntelligenceDb AS c2

CASE
  WHERE WITH .c1.countryCode AND
         .c1.countryCode = .c2.countryCodeIsoAlpha3
  GENERATE
    BUILD {
      .country           : .c1.country,
      .indicator         : .c1.indicator,
      .yearWeek          : .c1.yearWeek,
      .population        : .c1.population,
      .cases             : .c1.cases,
      .countryCode       : .c1.countryCode,
      .countryCodeIsoAlpha2 : .c2.countryCodeIsoAlpha2
    };

8. LOOKUP FROM WEB
FOR EACH WITH .countryCode
  CALL "http://api.geonames.org/countryInfoJSON"
    + "?formatted=true&lang=en"
    + "&country=" + .countryCodeIsoAlpha2
    + "&username=paolofosci&style=full";

9. EXPAND
UNPACK WITH .data.geonames
  ARRAY .data.geonames TO .geonames;

10. FILTER
CASE
  WHERE WITH .geonames, .source AND
         WITH .source.population
  GENERATE
    BUILD {
      .country           : .source.country,
      .indicator         : .source.indicator,
      .yearWeek          : .source.yearWeek,
      .population        : .source.population,
      .cases             : .source.cases,
      .countryCode       : .source.countryCode,
      .countryCodeIsoAlpha2 : .source.countryCodeIsoAlpha2,
      .capital           : .geonames.item.capital
    }
  WHERE WITH .geonames, .source AND
         WITHOUT .source.population
  GENERATE
    BUILD {
      .country           : .source.country,
      .indicator         : .source.indicator,
      .yearWeek          : .source.yearWeek,
      .population        : TO_INT(.geonames.item.population),
      .cases             : .source.cases,
      .countryCode       : .source.countryCode,
      .countryCodeIsoAlpha2 : .source.countryCodeIsoAlpha2,
      .capital           : .geonames.item.capital
    };

11. SAVE AS euMostRecentCovidData@softIntelligenceDb;

```

Specifically, the documents that were obtained by Line 6 in Listing 2 must be completed with the name of the capital of each country and, if missing, with the population. The script is presented hereafter.

```

{
  "cases"           : 1011,
  "country"         : "Luxembourg",
  "countryCode"     : "LUX",
  "countryCodeIsoAlpha2" : "LU",
  "indicator"       : "cases",
  "population"      : 645397,
  "yearWeek"        : "2023-13"
}

```

Fig. 9. Example of document in the temporary collection generated by Line 7 in Listing 3.


```

{
  "timestamp" : "2023-04-08T02:57:29.138",
  "url"       : "... http://api.geonames.org/countryInfoJSON?...&country=LU&..."
  "source"    : {
    "cases"           : 1011,
    "country"         : "Luxembourg",
    "countryCode"     : "LUX",
    "countryCodeIsoAlpha2" : "LU",
    "indicator"       : "cases",
    "population"      : 645397,
    "yearWeek"        : "2023-13"
  }
  "data"      : {
    "geonames" : [
      {
        "areaInSqKm"   : "2586.0",
        "capital"      : "Luxembourg",
        "continent"    : "EU",
        "continentName" : "Europe",
        "countryCode"  : "LU",
        "isoAlpha3"    : "LUX",
        "countryName"   : "Luxembourg",
        "currencyCode"  : "EUR",
        "population"    : "607728",
        ... // other less interesting fields
      }
    ]
  }
}

```

Fig. 10. Example of document generated by Line 8 in Listing 3.

- The `JOIN OF COLLECTIONS` instruction on Line 7 actually extends documents with the most recent indicators for each country with the country codes in the ISO *alpha-2* and ISO *alpha-3* formats. Remember that they are described in the `countryCodes` collection already present in the `softIntelligenceDb` database (see Sect. 5.2 and Fig. 4). We do not explain the instruction in details; the resulting collection contains exactly the same document in the input temporary collection, but extended with the `countryCodeIsoAlpha2` field (ISO *alpha-2* code), as illustrated in Fig. 9, where we report the same document depicted in Fig. 8 extended with the country codes.
- In order to add the name of the capital and, if missing, the population of each country, it is necessary to contact an external web service. This is done by the `LOOKUP FROM WEB` instruction on Line 8.

This statement has been introduced in [11]: for each document in the input temporary collection that satisfies the `FOR EACH` clause, the URL composed in the `CALL` clause is called. Specifically, a web service provided by GeoNames is called, providing the value of the `countryCodeIsoAlpha2` field to the `country` parameter of the URL (which requires an ISO *alpha-2* country-code to select a country).

The corresponding output document is structured as illustrated in Fig. 10: the `timestamp` field denotes the time the web services was contacted, while the `url` field denotes the full URL; the `source` field contains the source document for which the call was done, while the `data` field contains the *JSON* documents received from the web service.

```

{
  "timestamp" : "2023-04-08T02:57:29.138",
  "url" : "http://api.geonames.org/countryInfoJSON? ...&country=LU&...",
  "data" : {},
  "source" : {
    "cases" : 1011,
    "country" : "Luxembourg",
    "countryCode" : "LUX",
    "countryCodeIsoAlpha2" : "LU",
    "indicator" : "cases",
    "population" : 645397,
    "yearWeek" : "2023-13"
  },
  "geonames" : {
    "position" : 1
    "item" : {
      "areaInSqKm" : "2586.0",
      "capital" : "Luxembourg",
      "continent" : "EU",
      "continentName" : "Europe",
      "countryCode" : "LU",
      "isoAlpha3" : "LUX",
      "countryName" : "Luxembourg",
      "currencyCode" : "EUR",
      "population" : "607728",
      ... // other less interesting fields
    }
  }
}

```

Fig. 11. Example of document generated by Line 9 in Listing 3.

In the specific case, notice that this document contains a single `geonames` field, which is an array of one single document that describes the country (GeoNames returns always an array of documents).

- Consequently, it is necessary to flatten the documents: this is done in Line 9 by the `EXPAND` instruction, that unnests the only document contained in the `geonames` array. Figure 11 reports the document that is obtained from the document reported in Fig. 10.
- The process is completed by the `FILTER` instruction on Line 9. It contains two `WHERE` branches: the first one specifies what to do if the `population` field was present in the original document; the second branch specifies what to do if the `population` field was missing in the source document.

The final result is the same and, as illustrated in Fig. 12, documents have been enriched with the name of the capital and, if it was missing, with the population of the country.

- The final collection of the scheduled pre-processing script is saved by Line 10 into the `euMostRecentCovidData` collection within the `softIntelligenceDb` database, ready to be queried by analysts.

5.4 Template for Soft Querying Data

Without soft querying, it is not possible to talk about *Soft Web Intelligence*. Indeed, Problem 1 asks for exploiting soft querying capabilities, because through them it is possible to evaluate conditions in an imprecise way and rank documents.

```

{
  "capital"           : "Luxembourg",
  "cases"             : 1011,
  "country"           : "Luxembourg",
  "countryCode"       : "LUX",
  "countryCodeIsoAlpha2" : "LU",
  "indicator"         : "cases",
  "population"        : 645397,
  "yearWeek"          : "2023-13"
}

```

Fig. 12. Example of document in the collection generated by Listings 2 and 3.

Listing 4 reports the $J\text{-CO-QL}^+$ template developed for this goal. It is executed by the $J\text{-CO-BATCH}$ tool in the $J\text{-CO}$ Framework. Since a template is parametric, analysts could use it with different values of its parameters, so as to cope with different needs while using the same $J\text{-CO-QL}^+$ code. Hereafter, we present it in details.

Listing 4. $J\text{-CO-QL}^+$ script: soft querying EU COVID-19 data.

```

1. CREATE FUZZY OPERATOR covidAlertFO
  PARAMETERS
    population TYPE INTEGER,
    cases      TYPE INTERGE
  PRECONDITION population > 0 AND
    cases > 0
  EVALUATE 1000 * 1000 * cases / population
  POLYLINE
    [ ( 0, 0.00), ( 30, 0.00), ( 100, 0.01),
      ( 200, 0.15), ( 750, 0.50), (1500, 0.80),
      (2500, 0.90), (4000, 1.00) ];

2. USE DB softIntelligenceDb
   ON SERVER jcodes 'http://127.0.0.1:17017';

3. GET COLLECTION euMostRecentCovidData@softIntelligenceDb;

4. FILTER
  CASE WHERE WITH .population, .cases
  GENERATE
    CHECK FOR FUZZY SET covidAlert
    USING covidAlertFO (.population, .cases)
  ALPHACUT ##alphaCutThreshold## ON covidAlert
  BUILD {
    .capital      : .capital,
    .cases        : .cases,
    .country      : .country,
    .indicator     : .indicator,
    .countryCode  : .countryCode,
    .countryCodeIsoAlpha2 : .countryCodeIsoAlpha2,
    .population   : .population,
    .yearWeek     : .yearWeek,
    .casesPerMillion : TO_INT (1000*1000*.cases/.population),
    .covidRisk    : MEMBERSHIP_TO (covidAlert)
  }
  DEFUZZIFY;

5. SAVE AS euCountryCovidRisk@softIntelligenceDb;

```

Defining a Fuzzy Operator. Line 1 of the script in Listing 4 creates a “fuzzy operator”, i.e., an operator that can be exploited to evaluate the degree of membership of a document to a fuzzy set [23].

Very shortly, a “fuzzy set” A in a universe U is a mapping $A : U \rightarrow [0, 1]$: given an item $x \in U$, $A(x) \in [0, 1]$ is the membership degree of x to A ; in other words, if $A(x) = 1$, then x fully belongs to A ; if $A(x) = 0$, then x does not belong at all to A ; if $0 < A(x) < 1$, then x partially belongs to A (the greater the value, the greater the way x belongs to A).

For example, given a car a , we could see that it is in the set of *Fast_Cars* if its maximum speed is greater than 200 km/h, i.e., $Fast_Cars(a) = 1$. But what happens if the maximum speed is 190 km/h? The membership degree could be $Fast_Cars(a) = 0.80$, meaning that the car is not so fast.

The instruction on Line 1 of the script in Listing 4 creates a fuzzy operator named `covidAlertFO`. It receives two formal parameters named `population` and `cases` (as integer numbers), whose values must be both greater than 0 (as stated by the PRECONDITION clause). If the precondition is true, the EVALUATE clause computes the number of “cases per million”; the result is used as x -axis value against the membership function defined by the POLYLINE clause; the function is depicted in Fig. 13: if the x -axis value is less than 0 cases per million, the corresponding 0 value is returned as membership degree; if the x -axis value is greater than 4000 cases per million, the corresponding value 1 is returned as membership degree. Notice that the shape of the curve enhances the membership when cases per million are greater than 30.

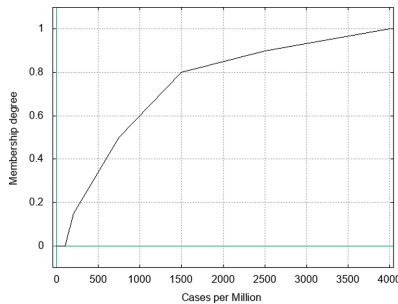


Fig. 13. Membership function for the `covidAlertFO` fuzzy operator.

Soft Querying. The remaining lines in the template reported in Listing 4 actually perform the soft query, based on the values of the macro parameter named `##alphaCutThreshold##`, which allows for specifying a minimum threshold for membership degrees, as illustrated hereafter.

Line 2 in Listing 4 connects to the `softIntelligenceDb` database, from which Line 3 acquires the content of the `euMostRecentCovidData` collection produced by the scheduled pre-processing script illustrated in Sect. 5.3. Remember that the collection contains the documents describing the last communicated cases of COVID-19 by each European country (see Fig. 12).

Line 4 is the only instruction necessary to perform the soft query, provided that the fuzzy operator has been defined (on Line 1). Hereafter, we explain each single clause.

- The WHERE condition selects all documents having both the `population` and the `cases` fields; by construction, all documents should have them both.
- Within the GENERATE section, the CHECK FOR FUZZY SET clause evaluates the membership degree of each selected document to a fuzzy set named

`covidAlert`. The membership is evaluated by means of the “soft condition” specified after the `USING` keyword.

Specifically, the soft condition exploits the `covidAlertFO` fuzzy operator (defined on Line 1): the operator is called passing the values of the `population` and of the `cases` fields. The membership degree provided by the fuzzy operator becomes the membership degree of the document to the `covidAlert` fuzzy set. As an example of this intermediate state, Fig. 14 depicts the document that derives from the one presented in Fig. 12: after the evaluation of the `CHECK FOR FUZZY SET` clause. A special `~fuzzysets` field is now present, which contains the `covidAlert` field whose value is the membership degree of the document to the homonym fuzzy set. In the *J-CO-QL*⁺ data model, the `~fuzzysets` field behaves as a key/value map, where the key is the name of a fuzzy set, while the value is the membership degree; this way, the membership degrees of the document to multiple fuzzy sets can be represented.

- The `ALPHACUT` clause specifies a minimum threshold for the membership to the `covidAlert` fuzzy set: only documents having a membership degree that is no less than the specified threshold are actually inserted into the output temporary collection.

In the template reported in Listing 4, the value is not specified: in its place, we find a “macro parameter” named `alphaCutThreshold` (reported in red color, for the sake of clarity). The value for the macro parameter is specified at execution time, as depicted in Listing 5 (a configuration file is submitted to *J-CO-BATCH* to provide macro parameters when a template is executed). In this case, the threshold of 0.5 is chosen, meaning that all documents having a membership degree to the `covidAlert` fuzzy set less than 0.5 are discarded.

Listing 5. Example of property file used for macro parameters in Listing 4.

```
alphaCutThreshold = 0.5
```

- The `BUILD` block of Line 4 in Listing 4 generates the final structure of documents that passes the `ALPHACUT` threshold. In particular, notice the `covidRisk` field, whose value is the membership degree to the `covidAlert` fuzzy set (it is provided by the `MEMBERSHIP_TO` built-in function). This field will denote the level of risk in the last (or more recent) week for the specified country.
- Finally, the `DEFUZZIFY` option “defuzzifies” the documents, i.e., it removes the `~fuzzysets` field from documents. As an example, consider the document depicted in Fig. 15, which is the final shape of the document reported in Fig. 14.

The template reported in Listing 4 ends with Line 5: the `SAVE AS` instruction saves the temporary collection generated by the instruction on Line 4 into the `euCountryCovidRisk` within the `softIntelligenceDb` database, for further exploitation by the analyst.

We can conclude that now the *J-CO* Framework can be actually exploited to perform Web Intelligence tasks directly working on *JSON* documents returned by Web sources. Furthermore, it is possible to exploit the support to soft querying provided by *J-CO-QL*⁺: this feature is unique in the panorama, in particular because it is provided by a stand-alone tool.

```

{
  "capital"           : "Lussemburgo",
  "cases"             : 1011,
  "casesPerMillion"   : 1566,
  "country"           : "Luxembourg",
  "countryCode"       : "LUX",
  "countryCodeIsoAlpha2" : "LU",
  "covidRisk"         : 0.806647778281193,
  "indicator"         : "cases",
  "population"        : 645397,
  "yearWeek"          : "2023-13",
  "~fuzzysets"        : {
    "covidAlert" : 0.806647778281193
  }
}

```

Fig. 14. Example of document generated after the CHECK FOR FUZZY SET clause of Line 4 in Listing 4.

```

{
  "capital"           : "Lussemburgo",
  "cases"             : 1011,
  "casesPerMillion"   : 1566,
  "country"           : "Luxembourg",
  "countryCode"       : "LUX",
  "countryCodeIsoAlpha2" : "LU",
  "covidRisk"         : 0.806647778281193,
  "indicator"         : "cases",
  "population"        : 645397,
  "yearWeek"          : "2023-13"
}

```

Fig. 15. Example of document generated by the template in Listing 4.

6 Conclusions

Moving from the initial idea, introduced over two decades ago to exploit data that can be found on the Word-Wide Web, to discover useful knowledge, in this paper we propose a modern interpretation of Web Intelligence, based on the current technological panorama in which data can be gathered from the Internet as *JSON* collections, possibly stored in *NoSQL* repositories and then processed by means of Soft Computing. This vision, that we named Soft Web Intelligence, has been enabled by the *J-CO* Framework, a software tool under development at the University of Bergamo (Italy), which is natively able to perform Soft Computing over data gathered from either different web sources or *JSON* document stores.

After defining our proposal for Soft Web Intelligence in details, and introducing the *J-CO* Framework, we presented a plausible case-study, to show that our vision is practically applicable. The case-study exploited real data about COVID-19 cases, made available by the Open-Data portal of ECDC, a European-Union Agency, and geographical information accessible through the web-services provided by *GeoNames*, a geographical database available on the Internet, to perform a Soft Web Intelligence activity in order to define real-time COVID-19 alert thresholds in European Countries.

The case-study demonstrates the feasibility of the concept of *Soft Web Intelligence*; nonetheless, this is due to the effectiveness of the *J-CO* Framework that, at the best of our knowledge, currently has no competitor platforms or tools that are able to enable (non-programmer) analysts to perform similar tasks.

While developing the *J-CO* Framework, we also carry on a continued research activity in order to improve performance and introduce new features. Currently, we are pursuing a line of evolution concerning the extension toward multi-grade fuzzy analysis, allowing users to define their own multi-grade fuzzy-set models and operators; furthermore, we are working on a general approach for defining soft aggregations.

References

1. Abir, B.K., Amel, G.T.: Towards fuzzy querying of NOSQL document-oriented databases. *DBKDA* **2015**, 163 (2015)
2. Bordogna, G., Capelli, S., Ciriello, D.E., Psaila, G.: A cross-analysis framework for multi-source volunteered, crowdsourced, and authoritative geographic information: the case study of volunteered personal traces analysis against transport network data. *Geo-spat. Inf. Sci.* **21**(3), 257–271 (2018)
3. Burini, F., Cortesi, N., Gotti, K., Psaila, G.: The urban nexus approach for analyzing mobility in the smart city: towards the identification of city users networking. *Mob. Inf. Syst.* **2018**, 6294872 (2018)
4. Burini, F., Cortesi, N., Psaila, G.: From data to rhizomes: applying a geographical concept to understand the mobility of tourists from geo-located tweets. In: *Informatics*, vol. 8(1), p. 1. Multidisciplinary Digital Publishing Institute (2021)
5. Butler, H., Daly, M., Doyle, A., Gillies, S., Hagen, S., Schaub, T., et al.: The geojson format. Internet Engineering Task Force (IETF) (2016)
6. Fosci, P., Marrara, S., Psaila, G.: Geosoft: a language for soft querying features within geojson information layers. In: Marchiori, M., Domínguez Mayo, F.J., Filipe, J. (eds.) *Web Information Systems and Technologies. WEBIST WEBIST 2020 2021*. LNBP, vol. 469, pp. 196–219. Springer International Publishing, Cham (2023). https://doi.org/10.1007/978-3-031-24197-0_11
7. Fosci, P., Psaila, G.: *J-CO*, a framework for fuzzy querying collections of *JSON* documents (Demo). In: Andreassen, T., De Tré, G., Kacprzyk, J., Legind Larsen, H., Bordogna, G., Zadrozny, S. (eds.) *FQAS 2021*. LNCS (LNAI), vol. 12871, pp. 142–153. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-86967-0_11
8. Fosci, P., Psaila, G.: Towards flexible retrieval, integration and analysis of Json data sets through fuzzy sets: a case study. *Information* **12**(7), 258 (2021)
9. Fosci, P., Psaila, G.: Intuitionistic fuzzy sets in J-CO-QL +? In: García Bringas, P., (et al). 17th International Conference on Soft Computing Models in Industrial and Environmental Applications (SOCO 2022). LNNS, vol. 531, pp. 134–145. Springer, Cham. https://doi.org/10.1007/978-3-031-18050-7_13
10. Fosci, P., Psaila, G.: Soft integration of geo-tagged data sets IN J-CO-QL+. *ISPRS Int. J. Geo Inf.* **11**(9), 484 (2022)
11. Fosci, P., Psaila, G., et al.: Towards soft web intelligence by collecting and processing json data sets from web sources. In: 18th International Conference on Web Information Systems and Technologies, pp. 302–313. No. 302, SCIPRESS (2022)
12. Han, J., Chang, K.C.: Data mining for web intelligence. *Computer* **35**(11), 64–70 (2002)
13. Kacprzyk, J., Zadrozny, S.: Soft computing and web intelligence for supporting consensus reaching. *Soft. Comput.* **14**(8), 833–846 (2010)

14. Medina, J.M., Blanco, I.J., Pons, O.: A fuzzy database engine for MongoDB. *Int. J. Intell. Syst. Online library* **37**, 5691–5764 (2022)
15. Mehrab, F., Harounabadi, A.: Apply uncertainty in document-oriented database (MongoDB) using F-xml. *J. Adv. Comput. Res.* **9**(3), 87–101 (2018)
16. Negash, S., Gray, P.: Business intelligence. In: Negash, S., Gray, P. (eds.) *Handbook On Decision Support Systems 2*, pp. 175–193. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-48716-6_9
17. Poli, V.S.R.: Fuzzy data mining and web intelligence. In: *International Conference on Fuzzy Theory and Its Applications (iFUZZY)*, pp. 74–79. IEEE (2015)
18. Psaila, G., Fosci, P.: Toward an analyst-oriented polystore framework for processing JSON geo-data. In: *International Conference on Applied Computing 2018, Budapest; Hungary*, 21–23 October 2018, pp. 213–222. IADIS (2018)
19. Psaila, G., Fosci, P.: J-CO: a platform-independent framework for managing geo-referenced JSON data sets. *Electronics* **10**(5), 621 (2021)
20. Psaila, G., Marrara, S., Fosci, P.: Soft querying GeoJSON documents within the j-co framework. In: *WEBIST*, pp. 253–265 (2020)
21. Reddy, P.V.S.: FUZZYALGOL: fuzzy algorithmic language for designing fuzzy algorithms. *J. Comput. Sci. Eng.* **2**(2), 21–24 (2010)
22. Yao, Y.Y., Zhong, N., Liu, J., Ohsuga, S.: Web Intelligence (WI) research challenges and trends in the new information age. In: Zhong, N., Yao, Y., Liu, J., Ohsuga, S. (eds.) *WI 2001. LNCS (LNAI)*, vol. 2198, pp. 1–17. Springer, Heidelberg (2001). https://doi.org/10.1007/3-540-45490-X_1
23. Zadeh, L.A.: Fuzzy sets. *Inf. Control* **8**(3), 338–353 (1965)
24. Zadeh, L.A.: A note on web intelligence, world knowledge and fuzzy logic. *Data Knowl. Eng.* **50**(3), 291–304 (2004)
25. Zadeh, L.A.: Web intelligence, world knowledge and fuzzy logic – the concept of web IQ (WIQ). In: Negoita, M.G., Howlett, R.J., Jain, L.C. (eds.) *KES 2004. LNCS (LNAI)*, vol. 3213, pp. 1–5. Springer, Heidelberg (2004). https://doi.org/10.1007/978-3-540-30132-5_1
26. Zhang, Y.Q., Lin, T.Y.: Computational web intelligence (CWI): synergy of computational intelligence and web technology. In: *World Congress on Computational Intelligence*, vol. 2, pp. 1104–1107. IEEE (2002)
27. Zhong, N., et al.: Web intelligence meets brain informatics. In: Zhong, N., Liu, J., Yao, Y., Wu, J., Lu, S., Li, K. (eds.) *WImBI 2006. LNCS (LNAI)*, vol. 4845, pp. 1–31. Springer, Heidelberg (2007). https://doi.org/10.1007/978-3-540-77028-2_1
28. Zuccala, A., Thelwall, M., Oppenheim, C., Dhiensa, R.: Web intelligence analyses of digital libraries: a case study of the national electronic library for health (NELH). *J. Doc.* **63**, 558–589 (2007)